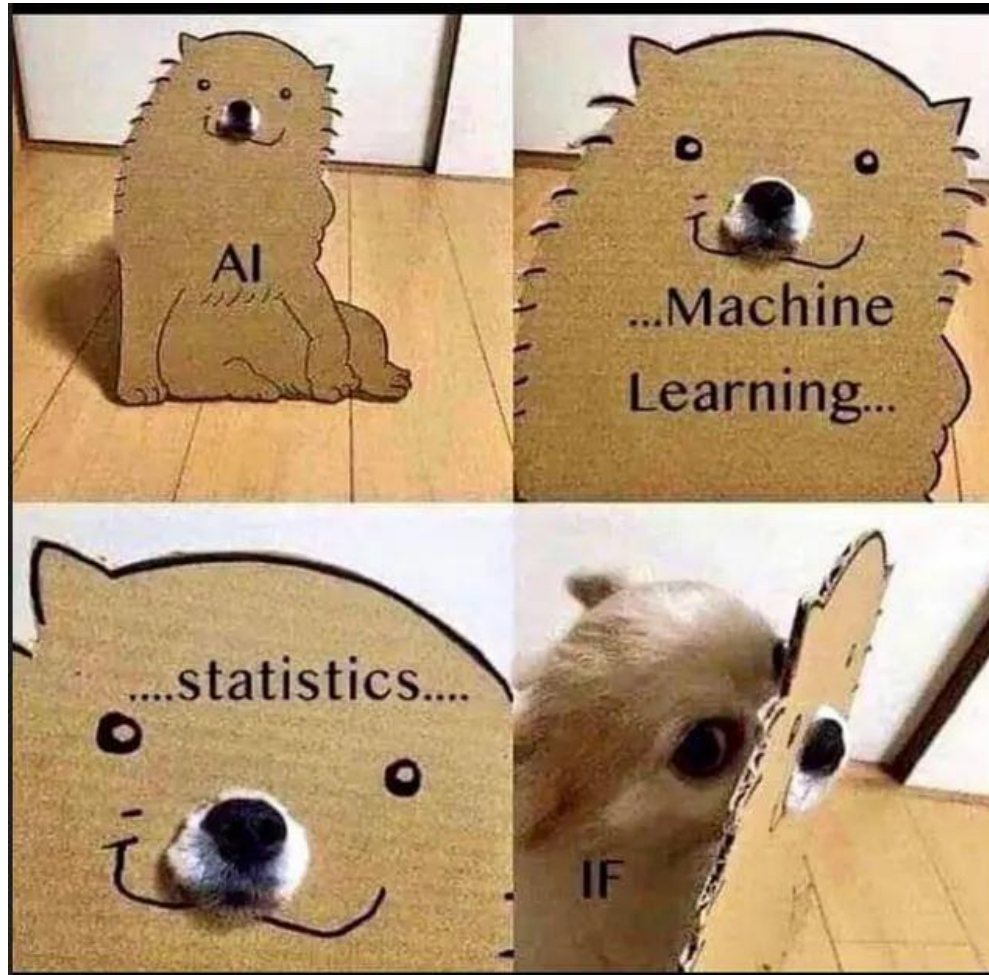


Bases del Entrenamiento

Daniel Barajas Higuera

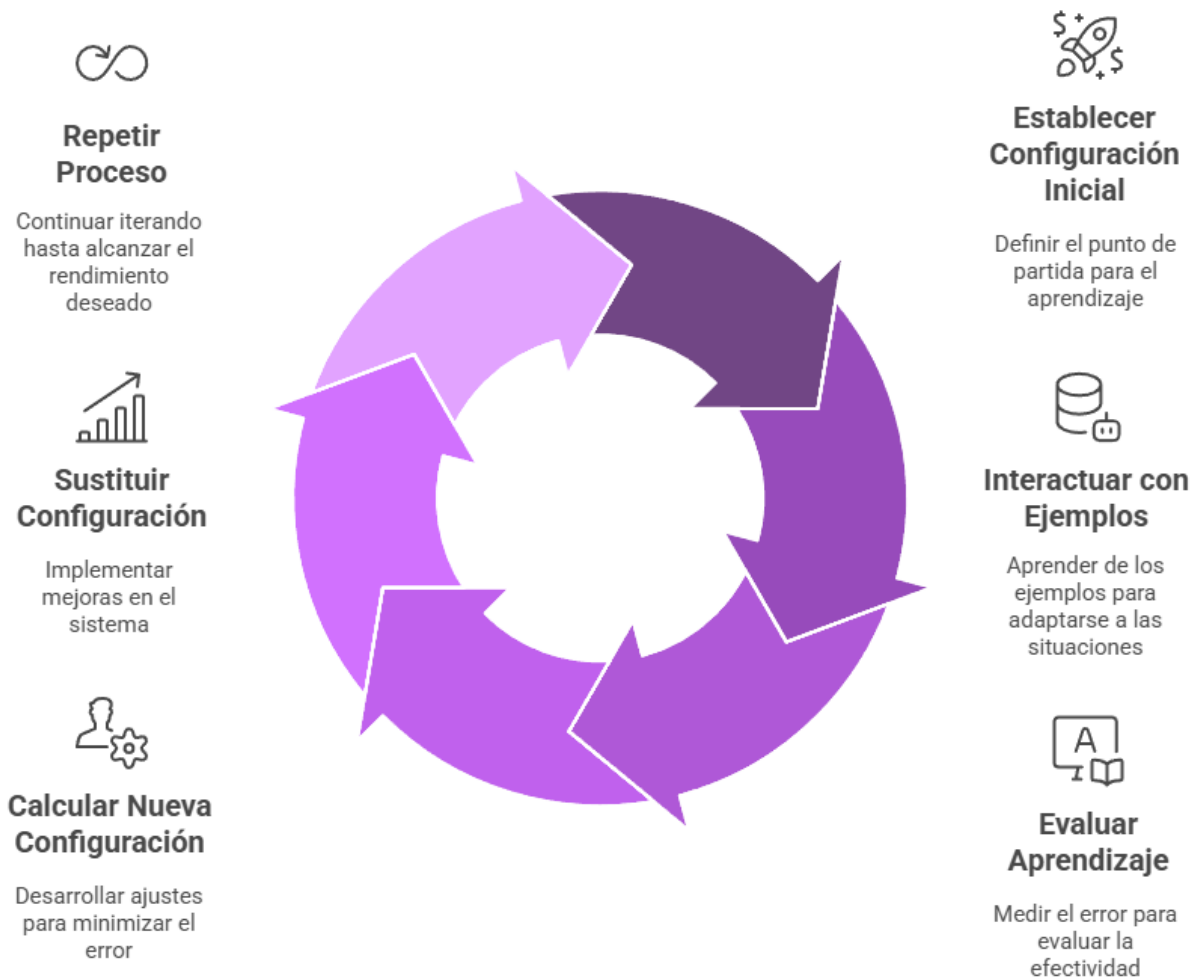


Machine Learning

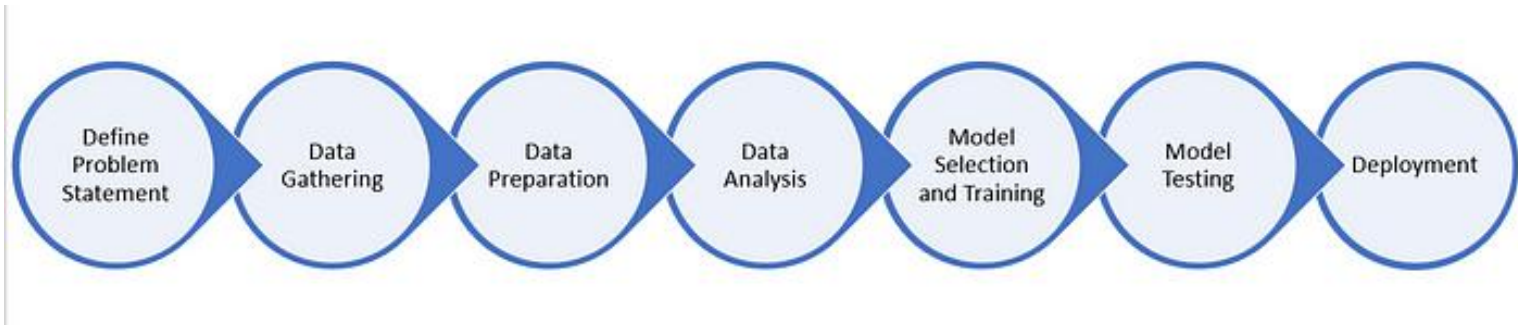


Ciclo de Aprendizaje

Ciclo de Aprendizaje



Ciclo de Aprendizaje

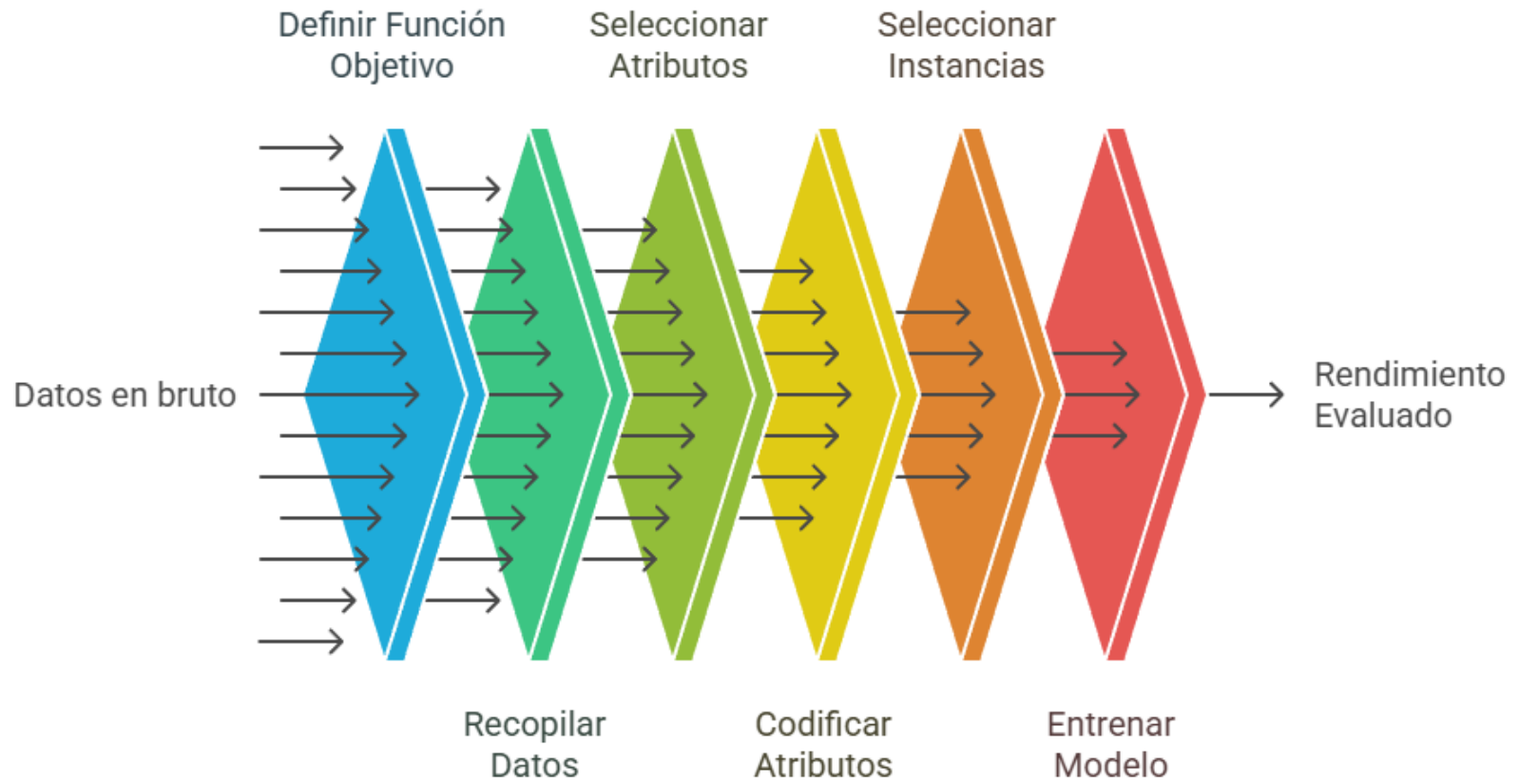


Ciclo de Aprendizaje

- ..  Working with Real Data
- ..  Look at the Big Picture
- ..  Get the Data
- ..  Discover and Visualize the Data to Gain Insights
- ..  Prepare the Data for Machine Learning Algorithms
- ..  Select and Train a Model
- ..  Fine-Tune Your Model
- ..  Launch, Monitor, and Maintain Your System

Ciclo de aprendizaje

Proceso de Preparación de Datos para el Aprendizaje Automático

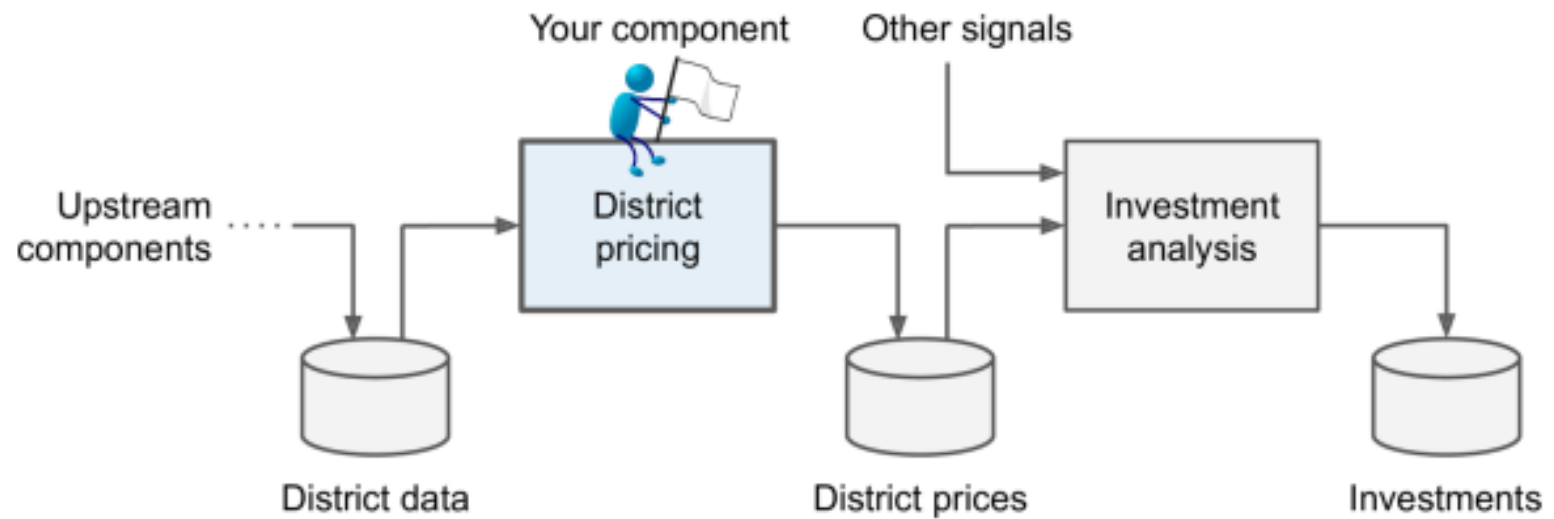


Datos reales

- Popular open data repositories
 - [UC Irvine Machine Learning Repository](#)
 - [Kaggle datasets](#)
 - [Amazon's AWS datasets](#)
- Meta portals (they list open data repositories)
 - [Data Portals](#)
 - [OpenDataMonitor](#)
 - [Quandl](#)
- Other pages listing many popular open data repositories
 - [Wikipedia's list of Machine Learning datasets](#)
 - [Quora.com](#)
 - [The datasets subreddit](#)

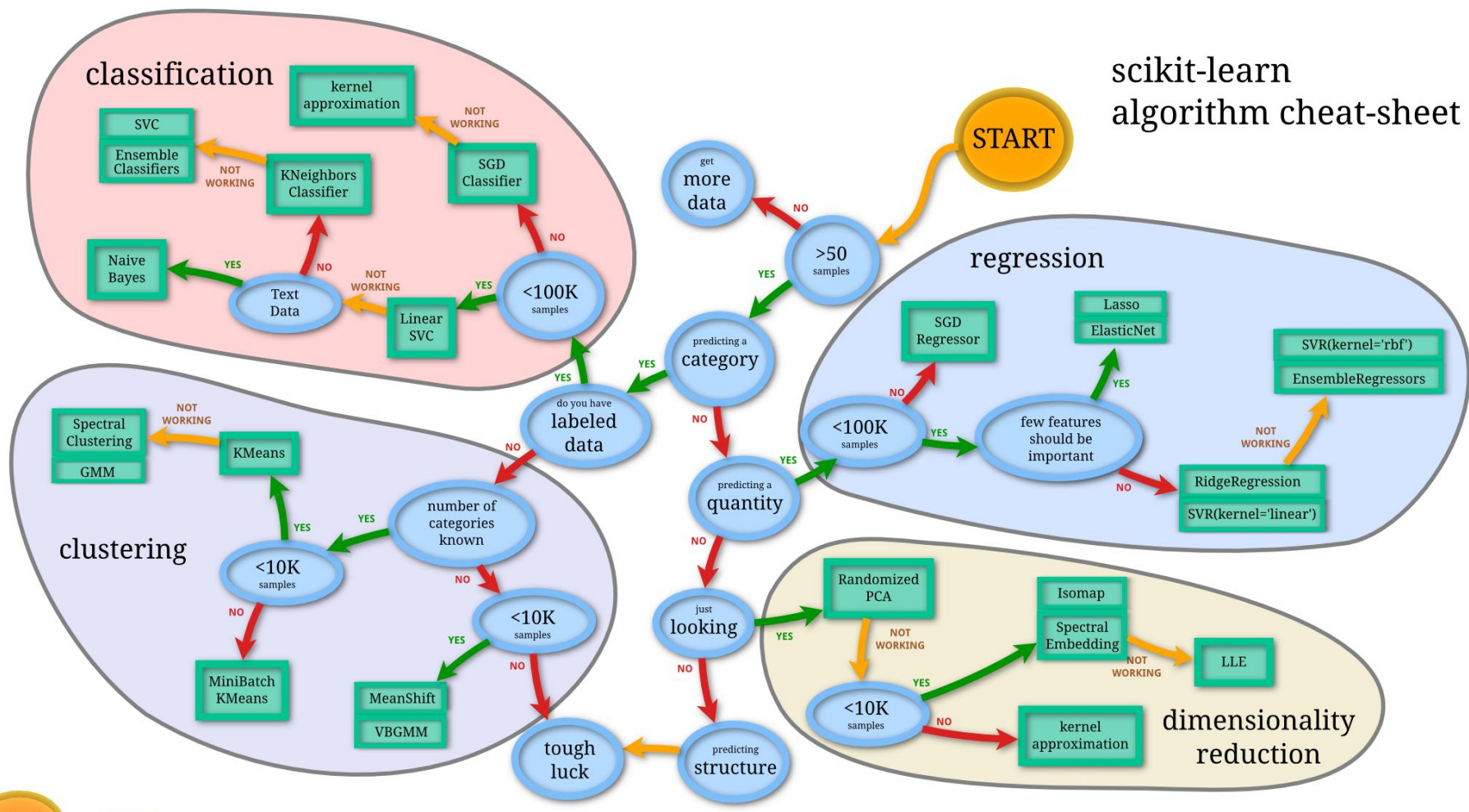


Entender el problema



Enmarcar el problema

scikit-learn
algorithm cheat-sheet



Méridas de eficiencia

$$Precision = \frac{TP}{TP + FP} \quad Recall = \frac{TP}{TP + FN}$$

$$RMSE(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(\mathbf{x}^{(i)}) - y^{(i)})^2}$$

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

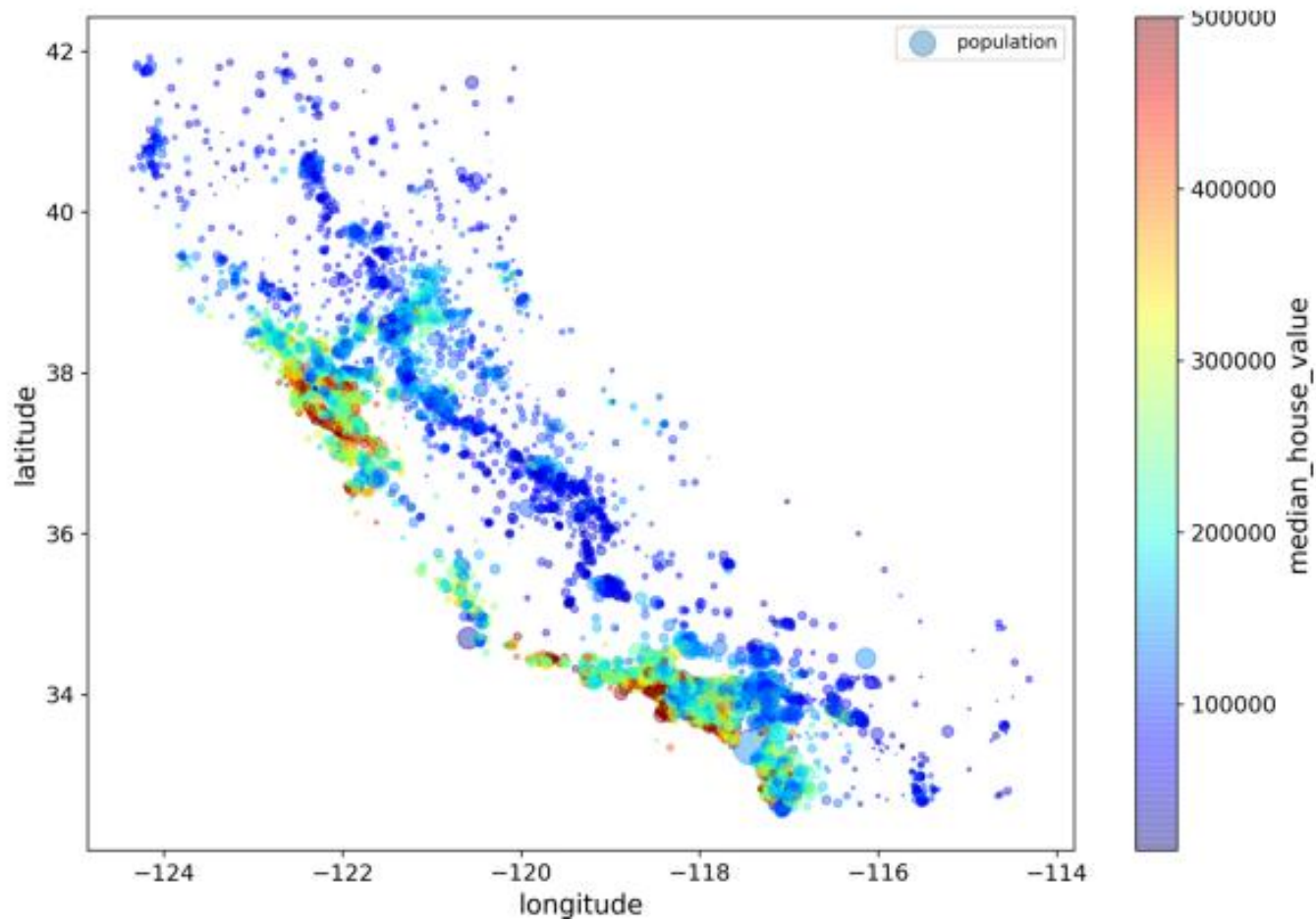
$$F1 \text{ Score} = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$MAE(\mathbf{X}, h) = \frac{1}{m} \sum_{i=1}^m |h(\mathbf{x}^{(i)}) - y^{(i)}|$$

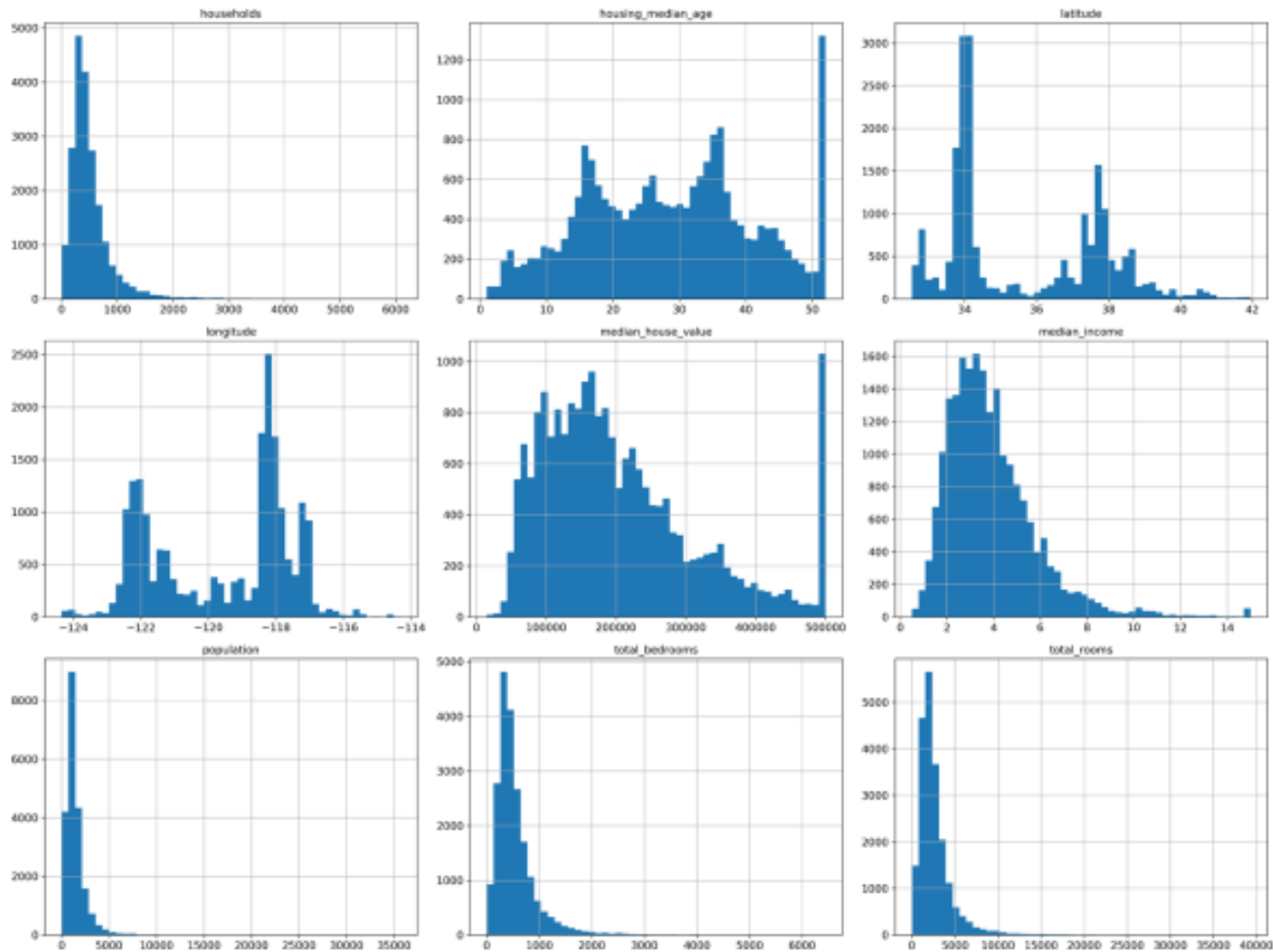
Conseguir Datos



Visualizar datos



Visualizar datos



Buscar correlaciones

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.687170
total_rooms           0.135231
housing_median_age    0.114220
households            0.064702
total_bedrooms        0.047865
population            -0.026699
longitude             -0.047279
latitude              -0.142826
Name: median_house_value, dtype: float64
```

Buscar correlaciones

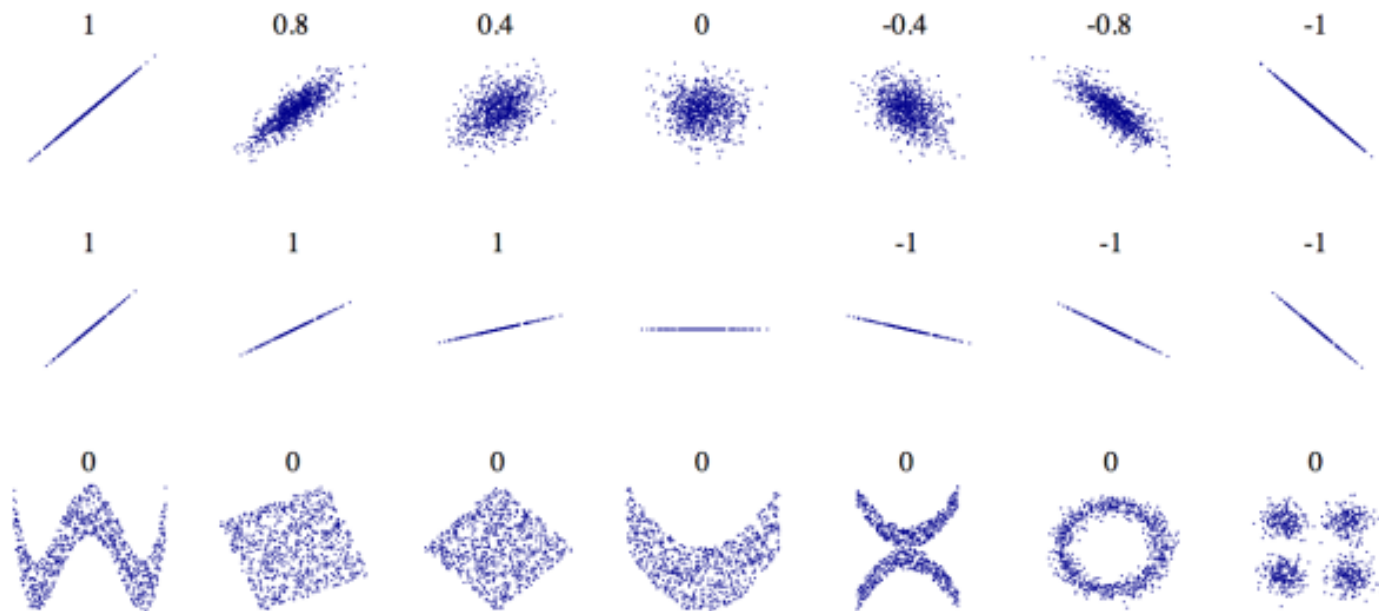
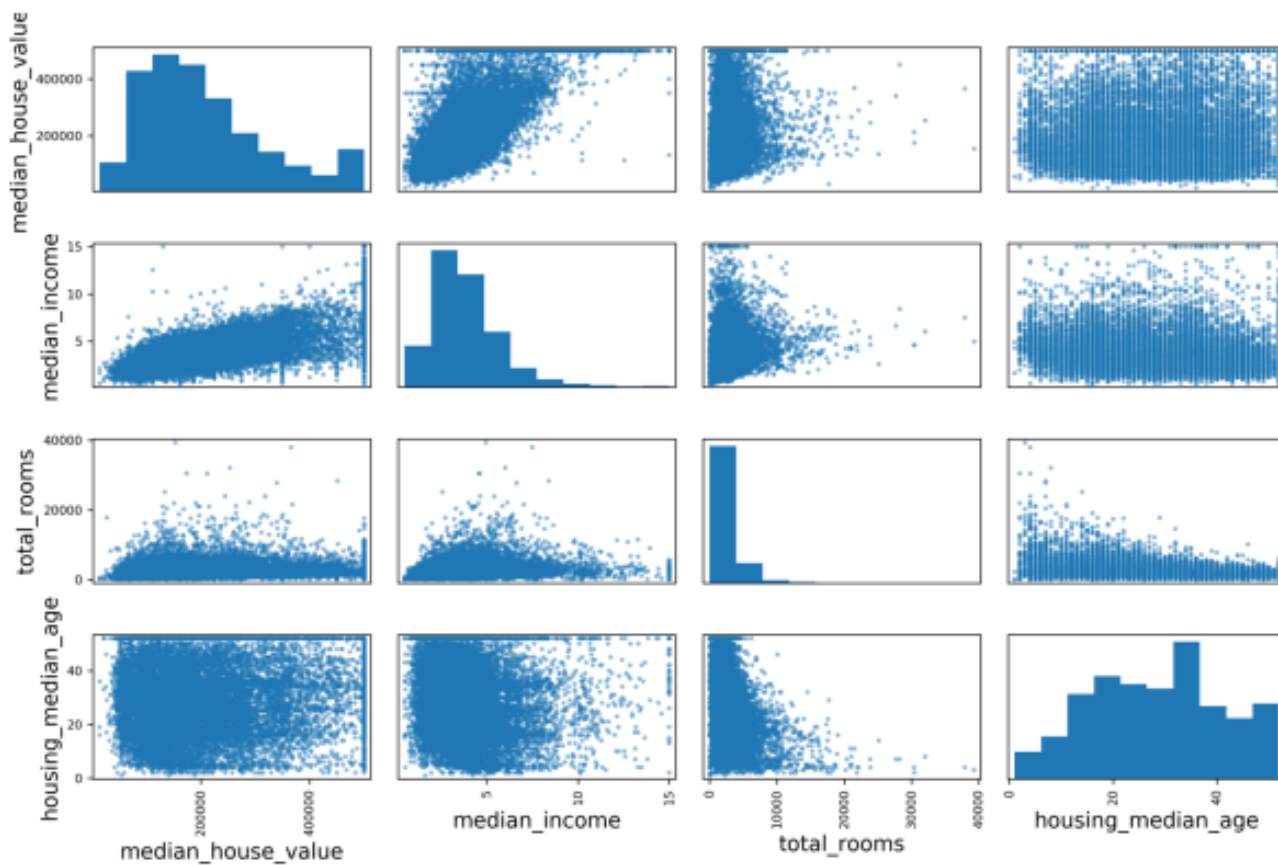


Figure 2-14. Standard correlation coefficient of various datasets (source: Wikipedia; public domain image)

Buscar correlaciones



Buscar correlaciones

Let's create these new attributes:

```
housing["rooms_per_household"] = housing["total_rooms"]/housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"]/housing["total_rooms"]
housing["population_per_household"]=housing["population"]/housing["households"]
```

And now let's look at the correlation matrix again:

```
>>> corr_matrix = housing.corr()
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value      1.000000
median_income           0.687160
rooms_per_household     0.146285
total_rooms             0.135097
housing_median_age      0.114110
households              0.064506
total_bedrooms          0.047689
population_per_household -0.021985
population              -0.026920
longitude               -0.047432
latitude                -0.142724
bedrooms_per_room       -0.259984
Name: median_house_value, dtype: float64
```

Preparar los datos

- Data Cleaning
- Handling Text and Categorical Attributes
- Custom Transformers
- Feature Scaling
- Transformation Pipelines

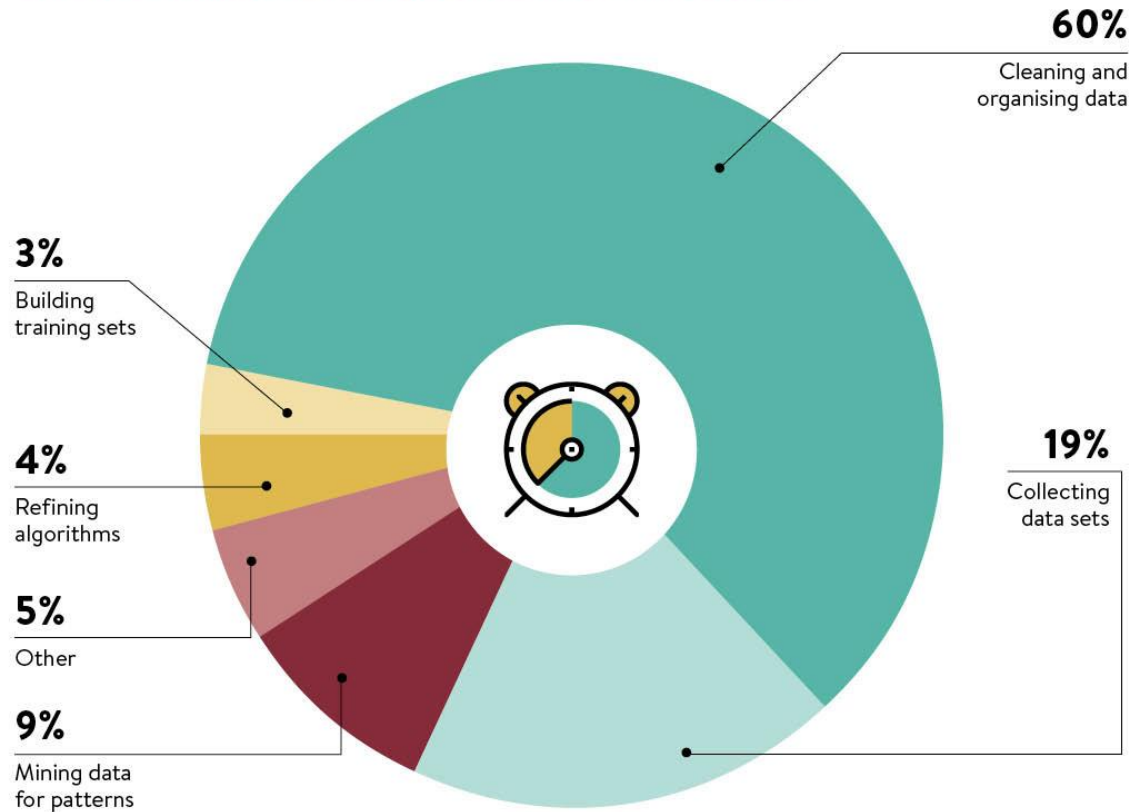


Preparar los datos



Preparar los datos

WHAT DATA SCIENTISTS SPEND THE MOST TIME DOING



Source: CrowdFlower 2016

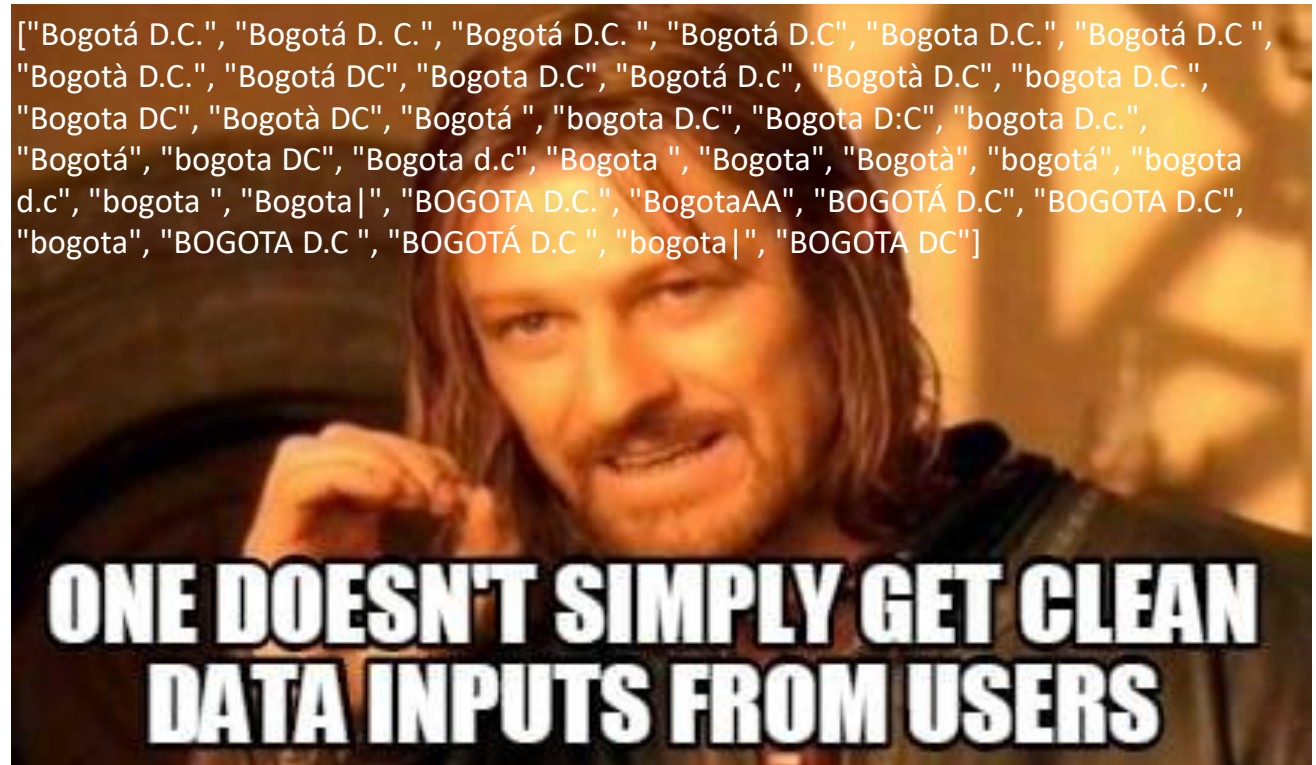
Preparar los datos

```
filter!(x -> !ismissing(x[2]), best_matches)[:"Bogotá D.C."]
```

37-element Vector{String}:

"Bogotá D.C."
"Bogotá D. C."
"Bogotá D.C. "
"Bogotá D.C."
"Bogotá D.C."
"Bogotá D.C. "
"Bogotá D.C."
"Bogotá D.C."
"Bogotá DC"
"Bogota D.C."
"Bogotá D.C."
"bogota D.C."
"Bogota DC"
"
"bogota d.c"
"bogota "
"Bogota|"
"BOGOTA D.C."
"BogotaAA"
"BOGOTÁ D.C."
"BOGOTA D.C."
"bogota"
"BOGOTA D.C "
"BOGOTÁ D.C "
"bogota|"
"BOGOTA DC"

["Bogotá D.C.", "Bogotá D. C.", "Bogotá D.C. ", "Bogotá D.C", "Bogota D.C.", "Bogotá D.C ",
"Bogotá D.C.", "Bogotá DC", "Bogota D.C", "Bogotá D.c", "Bogotá D.C", "bogota D.C.",
"Bogota DC", "Bogotá DC", "Bogotá ", "bogota D.C", "Bogota D:C", "bogota D.c",
"Bogotá", "bogota DC", "Bogota d.c", "Bogota ", "Bogota", "Bogotá", "bogotá", "bogota
d.c", "bogota ", "Bogota|", "BOGOTA D.C.", "BogotaAA", "BOGOTÁ D.C", "BOGOTA D.C",
"bogota", "BOGOTA D.C ", "BOGOTÁ D.C ", "bogota|", "BOGOTA DC"]



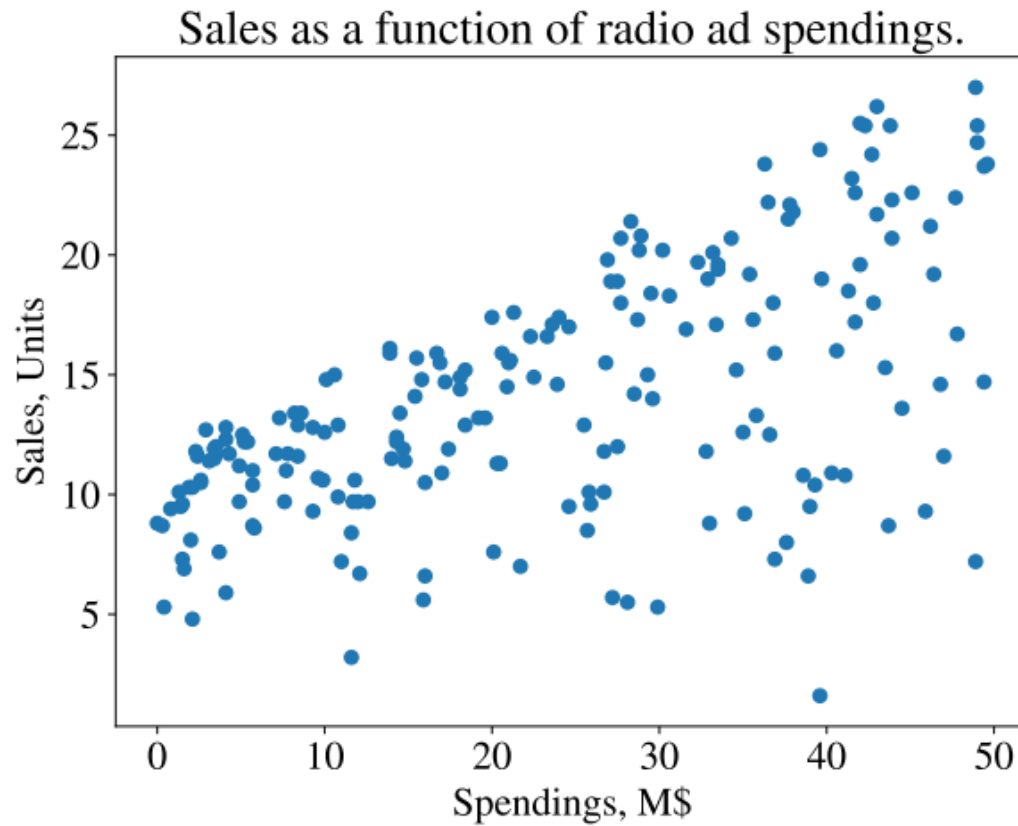
Entrenar el modelo

1. Una función de pérdida (loss function)
2. Un criterio de optimización basado en la función de pérdida (como una función de costo, por ejemplo)
3. Una rutina de optimización que utiliza los datos de entrenamiento para encontrar una solución al criterio de optimización.

Seleccionar y entrenar el modelo

```
from sklearn.linear_model import LinearRegression  
  
lin_reg = LinearRegression()  
lin_reg.fit(housing_prepared, housing_labels)
```

Ejemplo

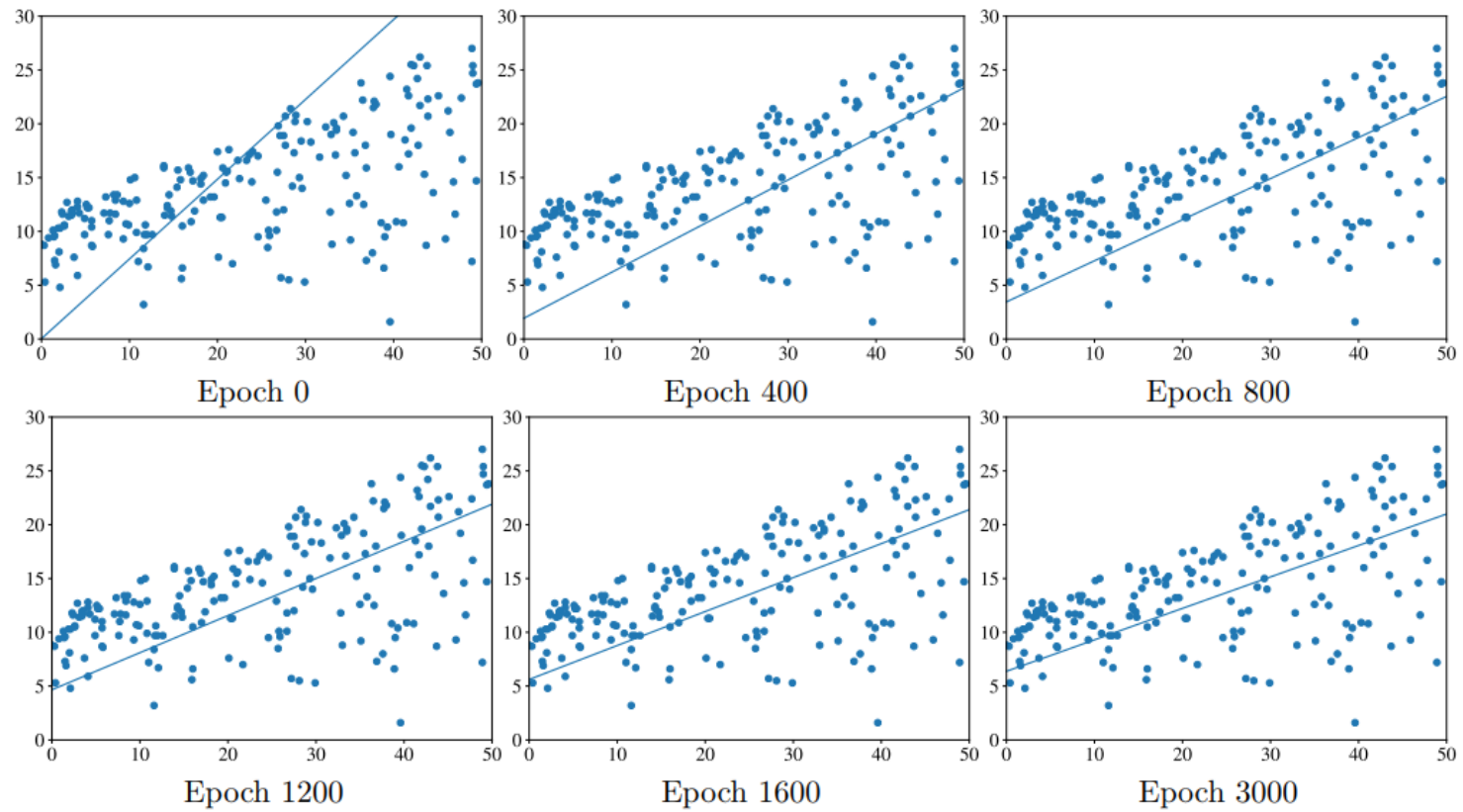


Loss Function

```
def avg_loss(spendings, sales, w, b):  
    N = len(spendings)  
    total_error = 0.0  
    for i in range(N):  
        total_error += (sales[i] - (w*spendings[i] + b))**2  
    return total_error / float(N)
```

```
epoch: 0 loss: 92.32078294903626  
epoch: 400 loss: 33.79131790081576  
epoch: 800 loss: 27.9918542960729  
epoch: 1200 loss: 24.33481690722147  
epoch: 1600 loss: 22.028754937538633  
...  
epoch: 2800 loss: 19.07940244306619
```

Evaluando el modelo



Ciclo de aprendizaje

1. Se escoge un modelo y parámetros.
2. Se dividen los ejemplos en 2 conjuntos:
 - Entrenamiento (80%): modificar los parámetros hasta que el error que se comete sea mínimo (bajar sesgo).
 - Prueba (20%): se calcula el error de generalización (bajo sobreajuste).

Problemas:

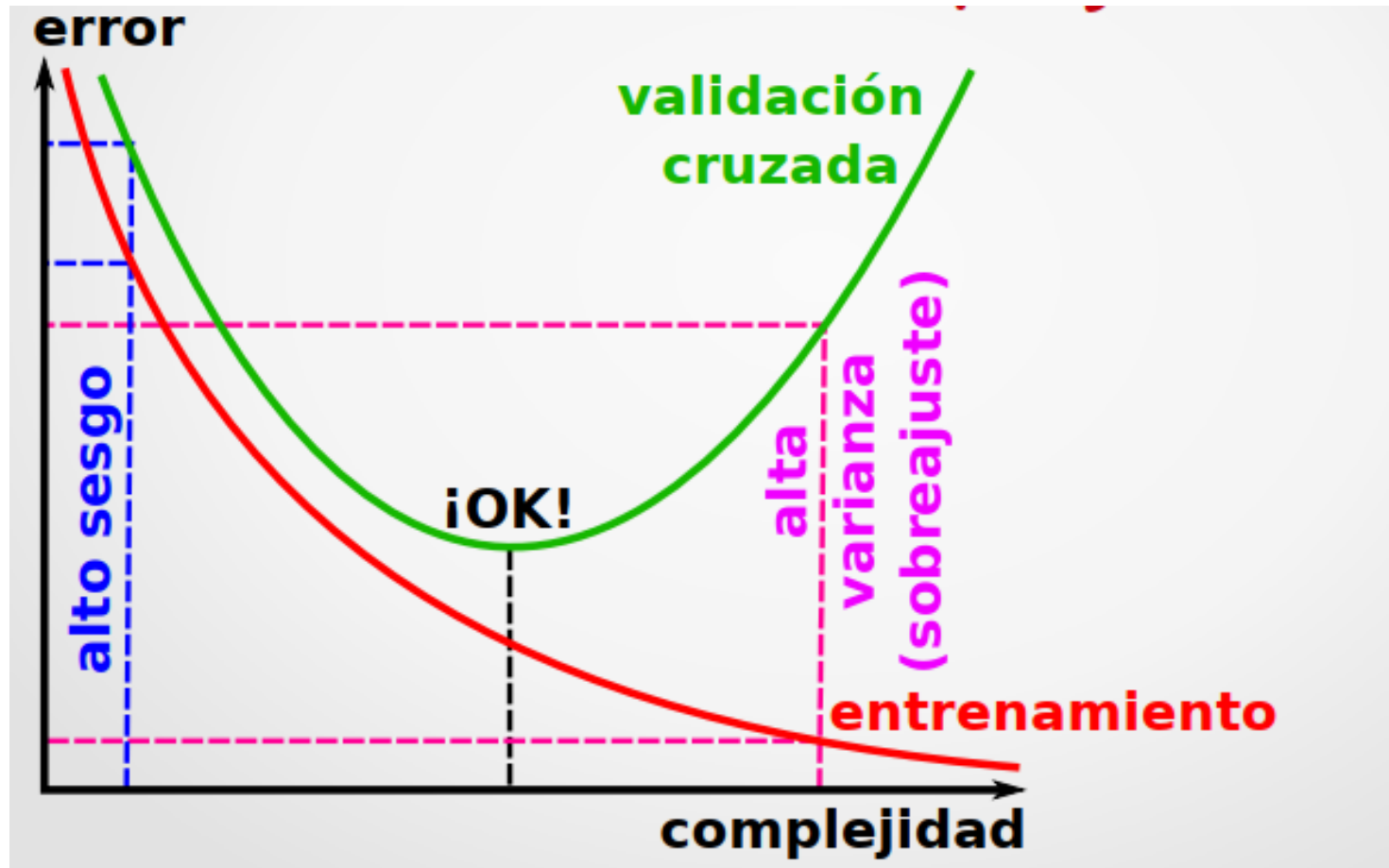
1. Sesgo (bias/underfitting).
2. Varianza (variance/overfitting).

Validación Cruzada

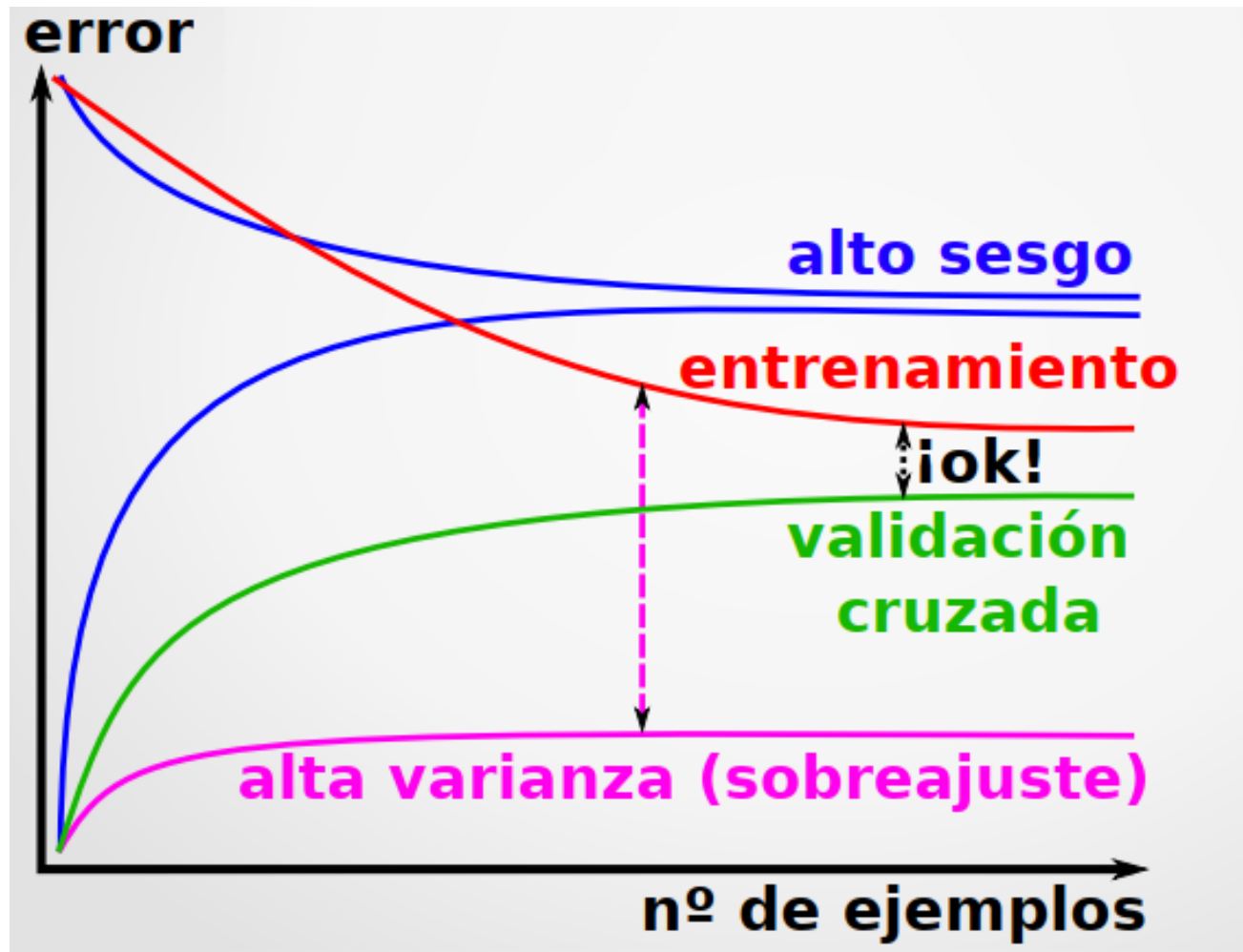
1. Se escoge un modelo.
2. Se dividen los ejemplos en 3 conjuntos:
 - Entrenamiento (60%): hasta que el error que se comete sea mínimo.
 - Validación (20%): evalúa parámetros y los modifica para minimizar error.
 - Prueba (20%): se calcula el error de generalización.

Evaluación: curvas de aprendizaje

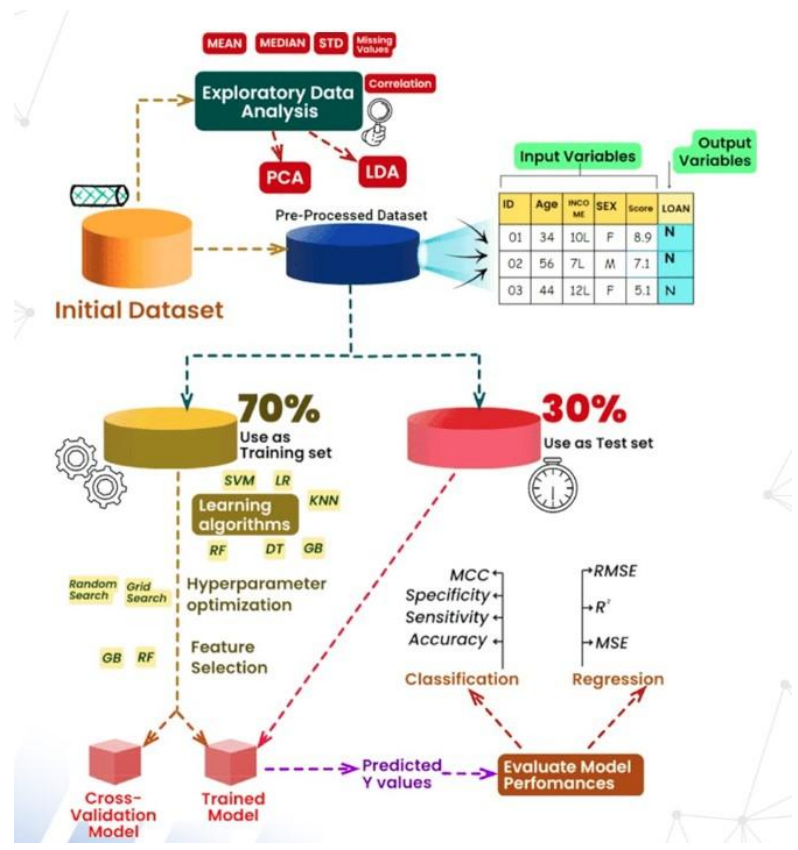
Error vs. Complejidad



Error vs. Número de ejemplos



Bases de entrenamiento



Bases de entrenamiento

